FIT3155
Advanced algorithms and data structures

Week 7 lecture notes by Taylor Kearney
Based on lecture slides by Arun Konagurthu

MONASH University

Faculty of Information Technology

May 5, 2023

Contents

1 Priority Queues . . . 2
2 Binary Heaps . . . 2
3 Fibonacci Heaps . . . 4
3.1 Representing Fibonacci heaps . . . 6
3.2 Fibonacci heap operations . . . 7
3.2.1 Min . . . 8
3.2.2 Merge . . . 8
3.2.3 Insert . . . 9
3.2.4 Extract-min . . . 10
3.2.5 Decrease-key . . . 16
3.2.6 Delete . . . 20
3.3 Why mark nodes? . . . 21
3.4 Summary . . . 21

1 Priority Queues

Using disjoint-sets we are able to divide elements within our set into subsets (equivalence classes) according to a given equivalence relation, but these are not the only interesting relations we might want to apply to our set. For instance, imagine you are tasked with scheduling a set of documents to be printed by a single printer. Some of the documents may contain a large number of pages and it might be desirable to print them last since they will require more time than the smaller jobs, or perhaps one of the documents is very important and so should be printed first. It should be obvious that this task - which arises naturally in many more interesting contexts - requires some assignment of *priorities* to the documents that dictate the *order* in which they will be printed. To achieve this we could assign a number, or *key*, from $[1, n]$ to each of the documents and decide that those documents assigned low numbers should be given precedence over those with higher numbers. In other words, if we select two elements from our set with keys α and β we simply determine if $\alpha < \beta$ in order to determine which of the two documents should be printed first. However, this relation ($<$) is not an equivalence relation (return to lecture 4 to double check this) and so we clearly need to develop a different kind of data structure to handle this problem.

You should recall from your previous studies that the problem above is most elegantly abstracted using *priority queues*, which in the broadest sense simply assign a priority to the elements in a set such that the element with the highest priority is served first. In addition, priority queues support several operations, which can dependent on the application, but typically one might need to:

- Insert a new element into the queue.
- Identify the element with the highest priority.
- Extract the element with the highest priority.
- Alter the priority of an existing element in the queue.

Hopefully it is clear how a priority queue supporting such operations could be used to solve scheduling problems like the contrived example above, but you should also recall that they can be used for sorting (heap sort), and feature prominently in many algorithms such as Prim's and Dijkstra's. Having defined the abstract notion of a priority queues we now turn our attention to their efficient implementation using heaps. In fact heaps are so commonly used to realise priority queues that the terms are often used interchangeably and hereafter our discussion will focus entirely on variants of the heap data-structure.

2 Binary Heaps

You are likely already familiar with a common heap variant known as a *binary heap*, but we will briefly review them here as they will serve as a good comparison point for the mergable variant that will occupy our focus for week 7.

Definition 1: Binary heaps

A binary heap is a binary tree that maintains two additional properties:

1. **Structure:** A binary heap is a complete binary tree, i.e. every level is full except possibly the last, which is always filled left-to-right.
2. **Order (min-heap):** The key of every node is less than or equal ($\leq$) to the keys of its children.

Each element in our heap is represented by a node in the tree, which has associated with it both a payload and a key. The order property above describes a min-heap[1] where the nodes with the smallest keys are assigned the highest priority. Furthermore, you should recall that min-heaps support the following operations:

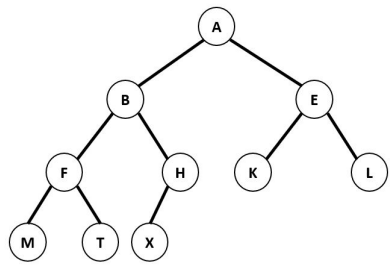
- **insert:** Insert a new element (containing a key and an associated payload) into the heap.
- **min:** Identify the minimum element in the heap.
- **extract-min:** Identify and remove the minimum element in the heap.

[1] We could equally consider max-heaps where the nodes with the largest keys are assigned the highest priority, but we will restrict ourselves to min-heaps here since translation between the two is not difficult.

- **decrease-key:** Decrease the key of an existing element in the heap.

Example 1: Binary heaps

Below is a binary (min-)heap containing 10 nodes. We can see that this heap satisfies the structure property as all the levels except the last (the leaf level) are completely filled. In addition, we can see that the order property is maintained as the key of every node is less than or equal to that of its children.



Recall that due to the regularity of binary heaps we can actually represent them using an array rather than explicitly representing binary tree itself. The array for this example can be seen below. For an element at position i its left child is stored at position $2i$, its right child is stored at position $2i+1$, and its parent is in position $\lfloor \frac{i}{2} \rfloor$. Note that if any of the previous formulas yield a position outside the bounds of the array this indicates that the corresponding element does not exist. For instance, the element H has no right child as indicated by the fact that $2i+1 = 2(5)+1 = 11$.

1	2	3	4	5	6	7	8	9	10
A	B	E	F	H	K	L	M	T	X

Binary heaps enable us to efficiently realise the aforementioned operations and the worst case time complexity of these operations is summarised in table 1. To complement these results the fact that all these operations can be implemented using simple array manipulations - which are incredibly fast on most systems - makes a binary heap an incredibly practical data structure with a wide range of applications.

Operation	Binary heap
`make-new-heap`	$O(1)$
`min`	$O(1)$
`extract-min`	$O(\log N)$
`decrease-key`	$O(\log N)$
`delete`	$O(\log N)$
`insert`	$O(\log N)$ worst case $O(1)$ amortized

Table 1: The worst case time complexity of binary heap operations. Note that the delete operation is usually implemented by simply calling a decrease-key followed by an extract-min, which is why it was omitted from the earlier list of operations.

If our heap only needs to support the operations listed in table 1 then binary heaps are typically a good choice[2]. However, what if we were to require an additional operation *merge* that simply merges two distinct heaps into one, are binary heaps able to implement this operation efficiently?

Let's consider two binary heaps H_1 and H_2 that contain N elements each. We know that both H_1 and H_2 will actually be represented using two arrays each containing N elements, and that representing the merged heap H will require an array of size $2N$. To complete the merge we need to copy the elements of H_1 and H_2 to this larger array in a such a way that the resulting array corresponds to the a valid representation of H. But clearly just the act of

[2]We will see later that there are some applications that only require these operations where binary heaps do not lead to the optimal solution.

copying these elements to our new array requires $O(N)$ time and we have yet to account for any of the comparisons necessary to determine the order in which this copying should occur.

When compared to the other binary heap operations merge is computationally expensive and we are left to ponder an interesting question. Is possible to design a data structure that supports a more efficient implementation of the merge operation **without** compromising on the computational cost of the other operations? As we shall soon see this is indeed possible and in pursuit of this goal we will eventually arrive at a heap capable of outperforming binary heaps in particular applications.

3 Fibonacci Heaps

Sources and recommended reading: Michael Fredman and Robert Tarjan, Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms, Journal of the ACM, 34(3) 596-615 (1987).
CLRS, Introduction to Algorithms (Chapter 19).

A *Fibonacci heap* is a mergable heap that offers a significant improvement in asymptotic complexity when compared to a binary heap in several important applications. In a binary heap we maintain a complete binary tree to preserve an $O(\log(N))$ bound on the height of the heap. We would like to leverage a similar idea when designing our new heaps, but instead we will use *binomial trees* as starting point.

Binomial trees and their properties are are not examinable and this section is only included to supplement your understanding of Fibonacci heaps.

Definition 2: Binomial trees

We define binomial trees recursively:

- The binomial tree of order 0 - denoted B_0 - is a singleton node.
- The binomial tree of order 1 - denoted B_1 - is constructed from two B_0 trees, by making one the child of the other.
- The binomial tree of order 2 - denoted B_2 - is constructed from two B_1 trees, by making one the child of the other.
- and so on...

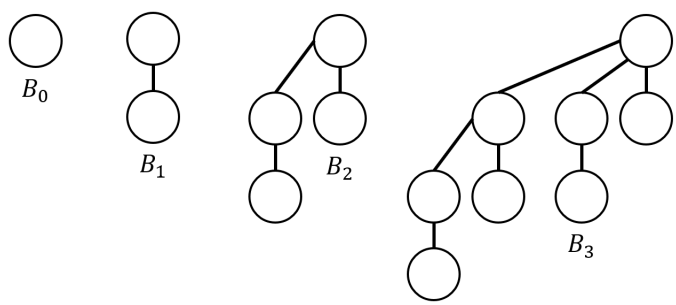


Figure 1: The binomial trees of order $0, 1, 2$ and 3 are denoted by B_0, B_1, B_2 and B_3 respectively.

Binomial tree properties

A binomial tree of order k (B_k) has the following properties,

1. B_k contains 2^k nodes.
2. B_k has height k.
3. The root of B_k has k subtrees as children.
4. Deleting the root node of B_k yields k independent lower order binomial trees $B_{k-1}, B_{k-2}, ..., B_0$.
5. The number of nodes at any given depth d is given by the *binomial coefficient* $\binom{k}{d} = \frac{k!}{(k-d)!d!}$, that is 'k-choose-d'.

It is an excellent exercise to prove the properties above formally (proof by induction is the natural choice), but they all - bar the last - follow rather straightforwardly from the definition of binomial trees. Let's consider each of them in turn.

Properties 1 and 2 can be be intuited by simply considering the recursive definition of binomial trees and noting that B_0 by definition contains $2^0 = 1$ node and has height 0. For instance, B_1 is constructed by making one B_0 tree the child of another and so must contain $2 \times 1 = 2^1$ nodes and have height 1. Similarly, B_2 is constructed by making one B_1 tree the child of another and so must contain $2 \times 2 = 2^2 = 4$ nodes and have height 2. This reasoning can easily be formalised into inductive proofs to show that these patterns hold true for a binomial tree of any order as claimed.

Similarly properties 3 and 4 become obvious once one examines a binomial tree. The root of a B_0 tree has 0 subtrees as it is a singleton node, the root of a B_1 tree has 1 B_0 tree as its subtree, which in turn implies that the root of a B_2 tree has two subtrees; one B_1 tree and one B_0 tree that belonged to the original B_1 tree. Finally, property 5 is responsible for the name *binomial* tree and is a little more difficult to intuit, but can be shown to follow from the recurrence relation for the binomial coefficient $\binom{k}{d} = \binom{k-1}{d-1} + \binom{k-1}{d}$.

End of non-examinable section.

Definition 3: Fibonacci heaps

A Fibonacci heap is a collection of rooted trees (a forest) where:

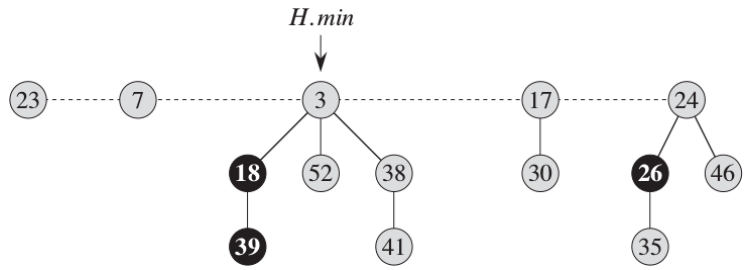
- **Order:** Each tree in the heap satisfies the heap property - i.e. the key of any node in the heap is less than or equal to ($\leq$) the keys of its children.
- **Structure:** Each tree is a binomial tree where each node is allowed to lose **at most** one child.

In addition, each node in the heap may be marked indicating that it has lost one of its children, or unmarked if it has yet to lose a child; we will return to the significance of these marks later. Finally, the heap maintains a reference (H_{min}) to the minimum element in the heap, which is always at the root level as each tree adheres to the min-heap property.

We will restrict our attention to min-heaps.

Example 2: Fibonacci heap

In the example shown below, 3 is the minimum element in the heap and the nodes with keys 18, 26, and 29 are marked as indicated by the darker shading. We can see that each tree is min-heap ordered and can be constructed from a binomial tree by removing at most one child from each node. For instance, the trees rooted at 23, 7, 24 and 17 are already binomial trees, while we can construct the tree rooted at 3 starting from a binomial tree of order 3 and removing a child from select nodes.



Precisely how the trees present in example 2 are produced will become clear after we discuss the operations supported by Fibonacci heaps, but first we need to discuss how we will represent our heaps.

3.1 Representing Fibonacci heaps

To enable efficient realisation of the operations supported by Fibonacci heaps we must opt for an explicit tree representation as an array based implementation will fundamentally limit the efficiency of our merge operation.

Fibonacci heap nodes

Any given node x in a Fibonacci heap is represented as an object (as shown in figure 2) that stores:

- A key ($x.key$) that is used to order the node within the heap.
- An optional payload ($x.payload$) that is auxiliary information associated with the node i.e. the information that is being stored within the node and ordered by the key.
- The degree of the node ($x.degree$), which is the number of children that the node has.
- If the node is marked or not ($x.mark$).
- A pointer to the node's parent ($x.parent$) (assumed to be *None* if the node is the root of a tree).
- A pointer to **any one** of the node's children ($x.children$) to enable traversal to the children list (assumed to be *None* if the node doesn't have any children).
- A pointer to its left **and** right siblings ($x.leftSibling$ and $x.rightSibling$) (assumed to point to the node itself if it has no siblings).

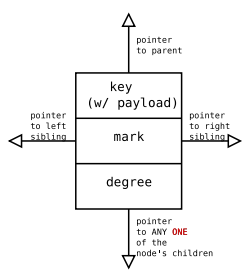


Figure 2: A depiction of a Fibonacci heap node.

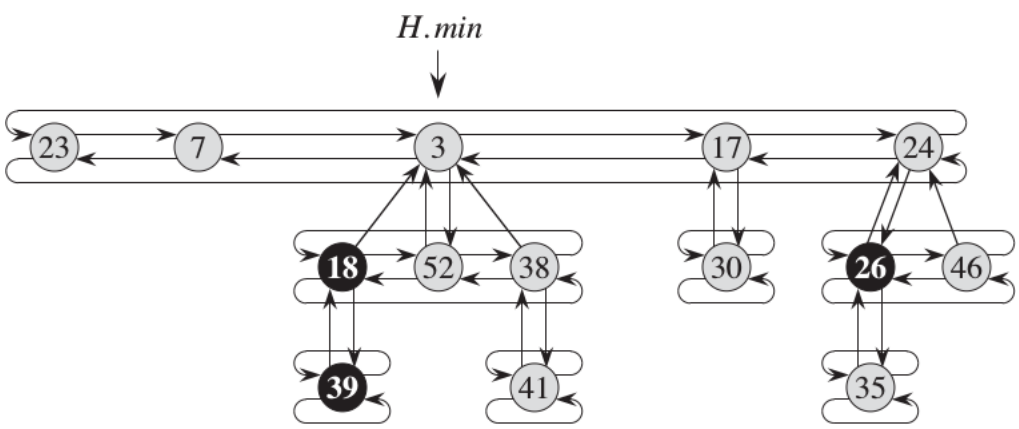
Since each node in the heap stores pointers to both its left and right siblings, each level in any given tree in our heap forms a circular doubly linked list. Similarly, we represent the *root list* - which consists of all the roots of the trees in the heap - in the same manner. The advantage of this representation is that it allows us to insert, or delete a node in any location in $O(1)$-time (assuming we are given the location as input). Furthermore, it enables us to join the elements of one list to another in $O(1)$-time and this fact will be important when it comes time to discuss the operations on Fibonacci heaps.

Example 3: Representing Fibonacci heaps

We can imagine each node in the heap below as being an object of the same class as the one depicted in figure 2. For instance, let x be the node with $key = 24$. We would store the following information in x:

- $x.key = 24$.
- $x.payload =$ some_payload, (the payload is dependent on the application and simply stores the object we are assigning priority $x.key$).
- $x.degree = 2$, as x has two children, 26 and 46.
- $x.marked = False$.
- $x.parent =$ None, as x is a root.
- $x.children =$ node_with_key_26.
- $x.leftSibling =$ node_with_key_17.
- $x.rightSibling =$ node_with_key_23.

As each level is represented using a circular doubly linked list we are able to access any node in that level by traveling left, or right from any other node in the same level. Thus, we only require a pointer to one of the children of a node in order to access all of its children. Similarly, we can access any element in the root list - and consequentially any element in the heap - through a pointer to any one of the nodes in the root list. In this sense $H_{\min}$ can be seen to serve a double purpose, as it allows us to easily access the minimum element in the heap while also providing us access to the other nodes in the heap. Finally, notice that when a level only contains a single node z, we consider the node to be its own siblings, i.e. $z.leftSibling = z$ and $z.rightSibling = z$, and examples of this can be seen in the nodes with keys 39, 41, 30, and 35 below.



3.2 Fibonacci heap operations

Recall that we wanted our mergable (min-)heaps to support the following operations:

- **insert:** Insert a new element (containing a key and an associated payload) into the heap.
- **min:** Identify the minimum element in the heap.
- **merge:** Merge two heaps H_1 and H_2 into one.
- **extract-min:** Identify and remove the minimum element in the heap.
- **decrease-key:** Decrease the key of an existing element in the heap.
- **delete:** Remove the specified element from the heap.

We will examine the implementation of each of these operations in turn beginning with the simplest.

3.2.1 Min

Finding the node with the smallest key in a Fibonacci heap is trivial since we maintain a reference to it (H_{min}).

Min

The pseudocode below implements this operation by simply returning the minimum key in a Fibonacci heap (H).

```
min(H):
    return H_min.key
```

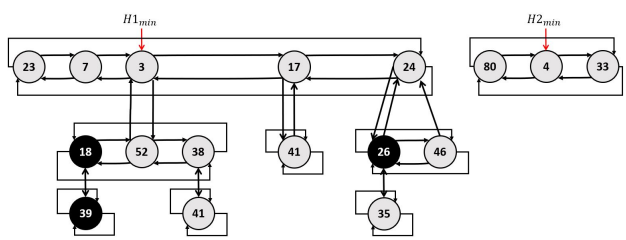
Clearly *min* for a Fibonacci heap is an $O(1)$-time operation.

3.2.2 Merge

It is both simple and efficient to merge two Fibonacci heaps. Moreover, the merge operation can be used as a foundation from which other operations are constructed.

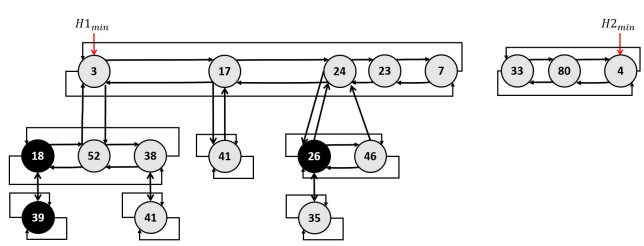
Example 4: Merging Fibonacci heaps

Let's consider merging the two heaps H_1 and H_2 whose minimum elements are 3 and 4 respectively, as shown below.

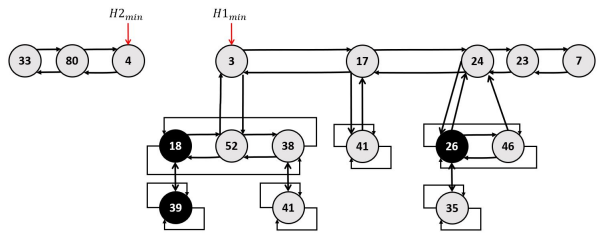


To merge these two heaps we must combine their root lists which can be accessed via $H1_{\text{min}}$ and $H2_{\text{min}}$ respectively. Conceptually we are trying to link two 'chains' that initially form two separate closed loops, which can each only be broken at one point.

Let's begin by rotating our root lists so that the minimum elements (the latch in our chains) are in a more suggestive position as shown below. Note that these are the exact same root lists as before, we have simply depicted them in a different manner.

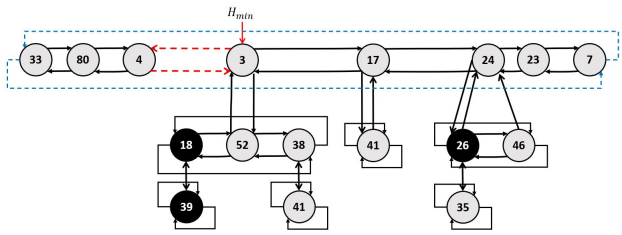


We'll choose to join the root lists by making $H2_{\text{min}}$ the left sibling of $H1_{\text{min}}$, however this is purely a convention and other implementations exist. Let's move H_2 to the left of H_1 (again this is simply for conceptual convenience) and 'break the chains' by removing the pointers on the right hand side of $H2_{\text{min}}$ and on the left hand side of $H1_{\text{min}}$.



Now all that remains is to make $H2_{\text{min}} = 4$ the left sibling of $H1_{\text{min}} = 3$ and then to join the other end of our 'chain'. This requires finding the previous left sibling of $H1_{\text{min}} = 3$ (7) and the previous right sibling of $H2_{\text{min}} = 4$ (33), and making these two nodes siblings. However, the order in which we set these links is important as we will overwrite references to the required nodes if we perform the updates in the wong order.

In this case we would first make the right sibling of 7 ($H1_{\text{min}}.leftSibling = 7$) 33 ($H2_{\text{min}}.rightSibling = 33$). Then we would make 7 the left sibling of 33 and at this point we have accounted for both the blue links below. Finally, we would make 3 the right sibling of 4, and 4 the left sibling of 3 - accounting for both the red links - and set H_{min} to which ever one of $H1_{\text{min}}$ and $H2_{\text{min}}$ had the smallest key.



Merge

To merge two Fibonacci heaps H_1 and H_2 into a single heap H we simply merge the root lists of H_1 and H_2 to form the root list of H as described by the pseudocode below.

```
merge(H1,H2):
    H1_min.leftSibling.rightSibling = H2_min.rightSibling
    H2_min.rightSibling.leftSibling = H1_min.leftSibling
    H2_min.rightSibling = H1_min
    H1_min.leftSibling = H2_min
    # Create a new Fibonacci heap
    H = newFibHeap(H1_min)
    # Set H_min to which ever of H1_min and H2_min has the smaller key
    H_min = min(H1_min,H2_min)
    return H
```

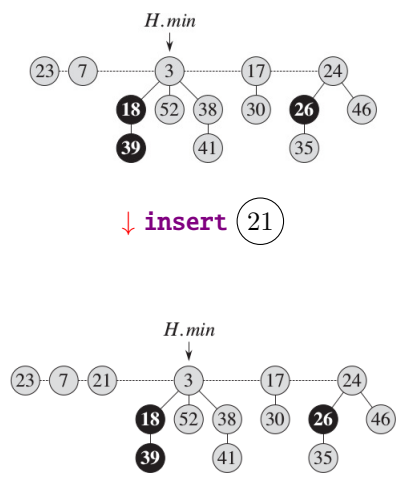
We are able to merge two Fibonacci heaps by simply updating 4 sibling pointers - which we can access in $O(1)$-time through $H1_{\text{min}}$ and $H2_{\text{min}}$ - yielding $O(1)$-time operation.

3.2.3 Insert

We can now lean on our ability to merge two heaps efficiently to implement insert. Our root list can contain multiple binomial trees of any given order (all the trees need to not even be binomial) and so inserting into a Fibonacci heap requires simply adding a single node into our root list.

Example 5: Inserting into a Fibonacci heap

Consider inserting a node with key 21 into the heap shown below. To do so we need to add our newly created node to the root list and as it is a circular doubly linked list the location at which we choose to add the node may seem arbitrary. But since we have $O(1)$-time access to the minimum element we choose to always insert nodes to the **left** of it so as to avoid wasting time with unnecessary traversal through the root list. Similar to the merge operation described in the previous section this requires updating 4 sibling pointers and then checking to see if the newly inserted node is the new minimum in our heap, which it is not in this case.



There are several equivalent implementations of the insert operation, but we'll leverage the merge operation to avoid redoing the required pointer gymnastics.

Insert

To insert a new element x into an existing Fibonacci heap H_1 containing N elements we:

1. Construct a new Fibonacci heap H_2 with x as its **only** element.
2. Run merge(H_1,H_2) to add x into the root list of H_1.

This process is also described in pseudocode below.

```
insert(key,payload,H1):
    nodeToInsert = newNode(key,payload)
    # Create a new heap containing just the newly created node
    H2 = newFibHeap(nodeToInsert)
    # Merging with the current heap makes the new node the left sibling of the current min
    H1 = merge(H1,H2)
    return H1
```

Merge for a Fibonacci heap is a constant time operation, which gives an $O(1)$-time worst case for insert compared to the $O(\log(N))$ worst-case and $O(1)$ amortised for inserting into a binary heap.

3.2.4 Extract-min

Since we don't perform any consolidation after merging two Fibonacci heaps we are able to implement both merge and insert in $O(1)$-time. However, the ability to identify and extract the minimum element is critical to all min-heap implementations and we cannot allow our root list to grow in size without bound if we are to support this operation efficiently. In general the effort required to extract the minimum element will depend on the length of the root list. However, due to the implementation of insert, our root list can contain N elements in the worst case, which will result in an $O(N)$-time extract-min. Despite this worst case we can still implement extract-min in $O(\log(N))$ amortised time.

To achieve this we consolidate our root list following each extract-min to minimise the number of elements it contains. In particular, our goal is to ensure that following an extract-min the root list contains at most one tree of

any given degree. It is important to note that while these trees may not all be strictly binomial trees (see section 3.2.5 for more details on this) and the degree of each tree plays an analogous roll to the order of a binomial tree. To see this recall that the root of binomial tree of order k (B_k) has exactly k children and so has degree k.

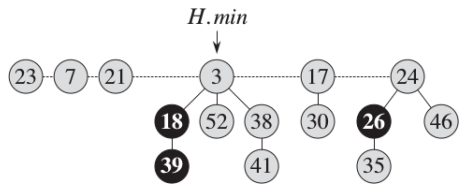
Extract-min

To extract the minimum element from a Fibonacci heap H we need to identify and delete the node in the root list with the smallest key and then consolidate the remaining trees. Performing this action requires the following steps.

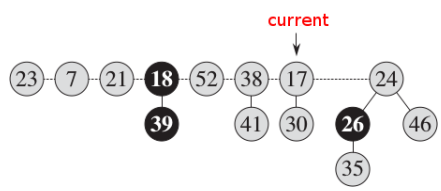
1. Remove H_{min} from the root list. This operation will produce a collection of orphaned children of varying degrees.
2. Create a new Fibonacci heap H_1 from the orphaned children produced by the previous step and merge it with the remainder of H.
3. Consolidate the resulting root list.

Example 6: Extracting the minimum from a Fibonacci heap

Let's explore the details of the extract-min operation by extracting the minimum element from the heap (H) below. First we must remove $H_{\min}$ (3) from the root list. Since the root list is a circular doubly linked list we can remove this element by making its left and right siblings, 21 and 17 respectively, siblings of each other and then setting the siblings of 3 to be itself.



However, this will remove the entire tree rooted at 3 from the heap and we need to reincorporate its children (18, 52 and 38). Fortunately these children are also connected in a circular doubly linked list and form a valid Fibonacci heap (H_1), which can be merged with the remainder of H. In this example we have set $H_{\min}$ to 17 (the right sibling of 3) following the extraction of the minimum and so H_1 is merged into the root list to the left of this element as described in section 3.2.2. Before continuing it is important to note that clearly 17 is not the minimum of the root list currently, but without traversing the entire list we cannot know where the true minimum resides. Thus, we set $H_{\min}$ to 17 **temporarily** to allow us to merge H_1 into H. Moreover, one must remember to set the parent pointers of the children of 3 to $None$ at some point during the operation as they now reside at the root level and this is most easily done before merging H_1 into H.

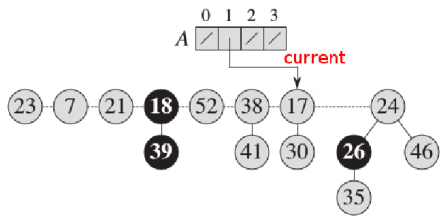


Having removed the minimum element from H we now need to consolidate our root list by merging nodes of equal degree. Since the trees within our root list are not ordered in any way and due to the nature of our representation there is no well defined 'start', or 'end' of our list. This complicates the process of consolidation slightly as we now need to keep track of the degree of each node we see and also where our traversal will start and end.

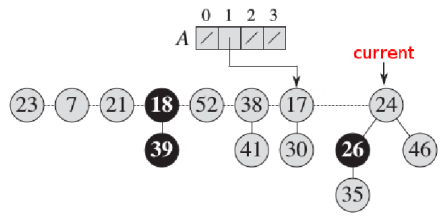
To this end, we will begin our traversal with 17 (indicated by the current pointer above) and make a rightward pass through the root list terminating once we've processed 38, which is the left sibling of 17 and should be tracked using another pointer which we'll call last. In addition, we will make use of an *auxilary array* (A) that is used to store the degree of the nodes we encounter during our traversal.

When we encounter a node x of some degree $d \geq 0$, we check position d in the auxilary array to determine if we have encountered another node of degree d previously. If $A[d]$ is not empty then a merge is required and x is merged with the node stored here. The resulting node - which will have degree $d + 1$ - then becomes the current node and the process repeats. On the other hand, if $A[d]$ is empty then we store a reference to x there and continue our traversal. In short, the auxilary array allows us to determine which trees in our root list need to be merged so that following the consolidation at most one node of each degree exists in the root list.

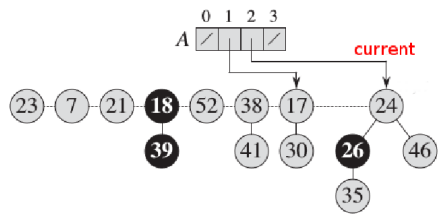
Let's now apply our new consolidation scheme to the example at hand. As stated previously, we'll start our traversal with 17 which has degree 1. Our auxilary array initially contains no nodes and so 17 is the first node of degree 1 that we've encountered and we simply store a reference to it in position 1 of our array as shown below.



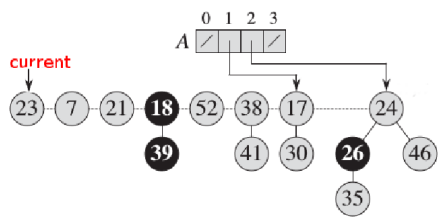
Having processed 17 we update current to point to 24, which is the right sibling of 17.



24 is a node of degree 2 and $A[2]$ is currently empty so we simply store a reference to 24 and continue our traversal rightwards.

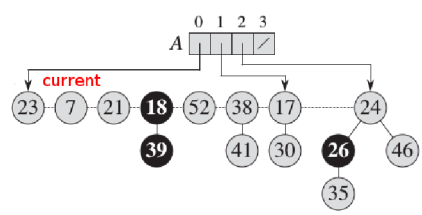


Now current becomes 23 (remember that our root list is circular) which has degree 0 and is stored at position 0 in the auxilary array.



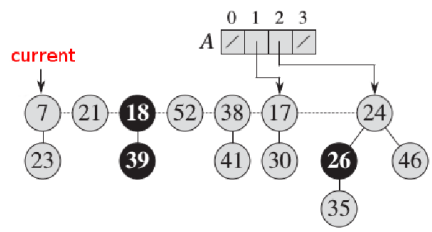
After storing 23 current is updated to 7, which has degree 0 and at this point we need to pause to consider a few important points. Firstly, 7 is the first node whose key is less than our current $H_{\min}$ (17) and so we need to update $H_{\min}$ to point to 7. In general, every time we encounter a new node during our traversal we need to check whether $H_{\min}$ requires updating, but this is the first time during our example that this has been the case.

Following this update we need to check $A[0]$ to see if we have encountered a node of degree 0 before. Since $A[0] = 23$ this indicates that we need to merge the trees rooted at 23 and 7 and we must do so such that the node with the smaller key (7) becomes the root of the resultant tree to ensure that our min-heap order is maintained.

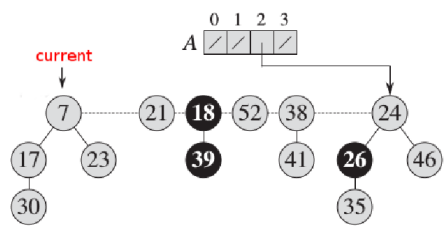


The aforementioned merger of 7 and 23 results in a tree of degree 1 rooted at 7 (note that 23 has been removed from the $A[0]$ slot) which now becomes the current node as shown below.

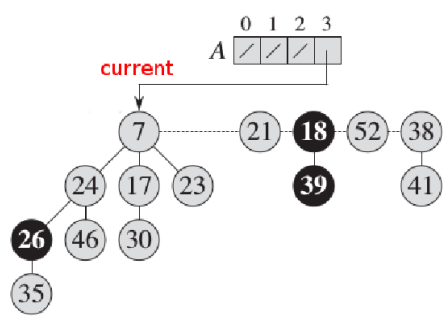
To process this new tree we examine $A[1] = 17$ and conclude that another merge is required.



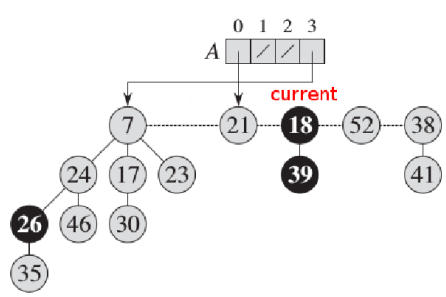
Just as before, the resulting tree is rooted at 7, which now has degree 2. Notice that the merge takes place at the location of the current node in the root list and **not** at the location of the node stored in A. The reason for this is that the root of the resulting tree will become current following the merge and we do not want to lose our progress through the root list by reverting to the location of a previously processed node. It should be stressed that the placement of the resulting tree is independent of whether the current node prior to the merge becomes the root or not. With this in mind we now inspect $A[2] = 24$, which triggers another merge.



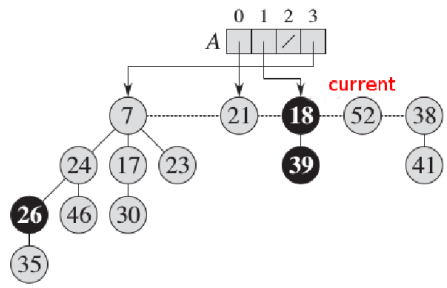
Merging 7 and 24 will yield the first tree of degree 3 that we have encountered, so we set $A[3] = 7$ and continue our traversal.



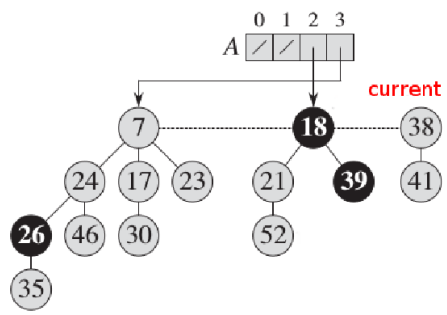
Next current becomes 21 whose degree is 0, so we set $A[0] = 21$ and move right to process 18.



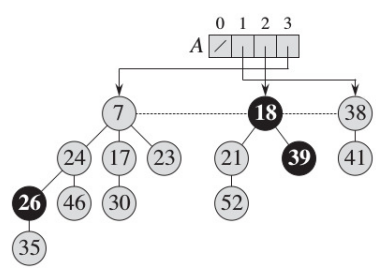
Similar to the previous iteration, 18 is a node of degree 1 and $A[1]$ is currently empty so we set $A[1] = 18$ and continue on to 52. Since $A[0] = 21$ we need to merge 21 and 52 to produce a tree of degree 1, which then must be merged with 18 resulting in a tree of degree 2.



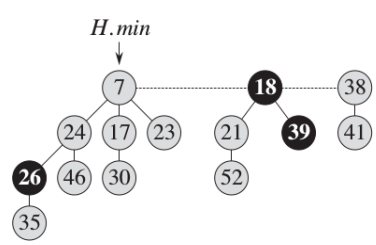
To proceed we set $A[2] = 18$ and then set current to 38. Before examining the final iteration, note that 18 was a marked node and although it wasn't treated specially here, there is one case that can arise that isn't covered by this example. In particular, if a marked root is ever made the child of another node during a merge we must **unmark** that node. The purpose of this will become clear shortly, but for now just remember this point when attempting to implement extract-min.



Finally, 38 is a node of degree 1 and $A[1]$ is empty so we set $A[1] = 38$. Normally we would then update current, but recall that we set 38 to be the last node and so once we have processed it we terminate our traversal.



Below we can see that following the consolidation $H_{\min}$ points to the minimum element in H (7) and the root list contains at most one node of any given degree; in this case we have a single node of degree 1, 2 and 3 and 0 nodes of all other degrees.



We'll close this section by briefly mentioning some important implementation details that are easy to overlook for extract-min.

Merging two trees in a Fibonacci heap

Firstly, one should consider how best to merge two trees, with roots r_1 and r_2, of the same degree as this operation was taken for granted above. Fortunately, the operation is simple enough and can be performed in constant ($O(1)$) time. If we assume that $r_1.key < r_2.key$, the main steps are:

- Make r_1 the parent of r_2.
- Add r_2 to the child list of r_1.
- Increment the degree of r_1.

Question 1: Merging two trees

Write pseudocode to merge two trees, with roots r_1 and r_2, of the same degree given the representation we have established here. Be sure to carefully consider the order in which you update the required pointers.

Finally, we conclude this section by noting that in the worst case extract-min is $O(N)$-time, but it is $O(\log(N))$ amortised. While we will leave the amortised analysis until week 8 we would be remiss to not at least acknowledge the fact that the base of the logarithm in this bound is actually the *golden ratio* $\phi = \frac{1+\sqrt{5}}{2}$. This is not a coincidence and is our first hint at the deeper connection between Fibonacci heaps and the Fibonacci sequence after which they are named. In addition, one can actually show that the degree of any node in a Fibonacci heap is $\leq \log_\phi(N)$ once all operations are supported and we will return to this point in the next section.

3.2.5 Decrease-key

We have yet to mention any specific application where Fibonacci heaps outperform their binary counterparts, and so far it seems they only have a distinct advantage in the merge operation which can be performed in constant time. However, Fibonacci heaps were originally designed to optimise *Dijkstra's shortest path algorithm*, which uses a priority queue (usually implemented as a min-heap) to find the shortest path from a source vertex to all other vertices in a directed graph with non-negative edge weights[3].

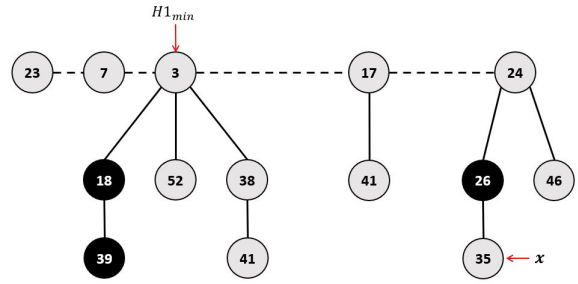
You should recall from your previous studies that when implemented using a binary heap, the worst case complexity of Dijkstra's is $O(|E|\log|V| + |V|\log|V|)$, where $|E|$ and $|V|$ denote the size of the input edge and vertex sets respectively. The $O(|V|\log|V|)$ contribution comes from the fact that we perform an extract-min - which is $O(\log(N))$ in the worst case for a heap with N nodes - for each vertex in our heap and our heap initially contains all of V. However, for dense graphs ($|E| = O(|V|^2)$) the extract-min term is dominated by the $O(|E|\log|V|)$ contribution from updating the distance to each vertex after we extract a node. These updates require decreasing the keys of vertices adjacent to the vertex we just extracted from the heap and in the worst case we must perform a decrease-key for every edge in the graph. As decrease-key is an $O(\log(N))$-time operation in the worst case for binary heaps, this yields total cost of $O(|E|\log|V|)$ for the distance updates over the algorithm. The purpose of Fibonacci heaps is to better this worst case complexity by reducing the cost of the decrease-key operation to $O(1)$ (amortised) allowing Dijkstra's to run in $O(|E| + |V|\log|V|)$.

In order to have our decrease-key run in better than $O(\log(N))$-time we can no longer 'bubble' the node up the heap if it violates our heap order. Instead we will leverage our 'lazy' merge - which defers consolidation to the next extract-min - to quickly recover our min-heap order. There are two cases that can occur and as usual we assume we are passed a reference to the node whose key we wish to decrease as our heap cannot be efficiently searched.

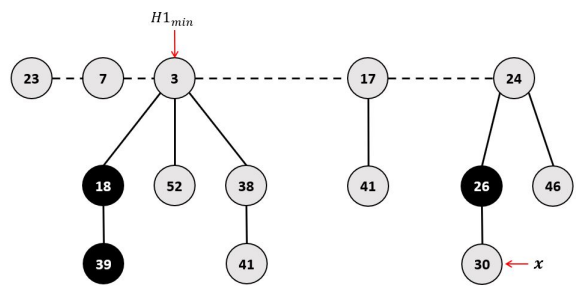
[3] While we'll focus on Dijkstra's here owing to our familiarity with it, Fibonacci heaps are applicable to several other applications.

Decrease-key case 1: The heap property <u>is not</u> violated

If decreasing the key of a node x does not violate the heap property, i.e. x's new key is still greater than or equal to $x.parent.key$, then no modifications need to be made to the structure of the heap.



↓ **decrease_key**(35 → 30)



If the decrease key operation **does** violate our heap property we need to restore order to our heap, however we want to do so in $O(1)$ (amortised) effort. To achieve this we first make use of the fact that we are able to have multiple trees of the same degree in our root list to quickly remove, or *cut* the offending node (x) from its tree before adding it back to the root list. Removing x and its associated subtree requires updating a constant number of pointers and so requires $O(1)$-time. Furthermore, as we decrease x's key we know that the tree rooted at x is still a Fibonacci heap and so we can simply merge it back into our root list which also requires $O(1)$-time.

Decrease-key case 2: The heap property <u>is</u> violated

If decreasing the key of a node x does violate the heap property, i.e. x's new key is now smaller than $x.parent.key$, then x (along with its subtree) needs to to be cut from its tree and promoted to the root list to restore min-heap order. Following this cut we need to update $H_{\min}$ if required, unmark x and mark the (previous) parent of x to indicate that it has now lost one child. However, it is possible for this parent to have already lost a child in which case it would already be marked. Furthermore, since each node can lose at most one child (revisit definition 3 if you need to) in this case we must also cut and promote the parent node. This cutting and promoting can then cascade up our tree and we'll address it separately yielding two subcases.

Case 2a
If (prior to cutting x) $y = x.parent$ is unmarked, i.e. $y.mark = false$, we cut and promote the subtree rooted at x to the root list - remembering to unmark x and update $H_{\min}$ if required - and mark y before concluding.

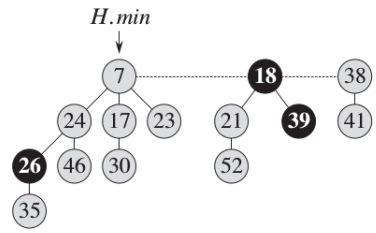
Case 2b
If (prior to cutting x) $y = x.parent$ is marked, i.e. $y.mark = true$, we cut and promote the subtree rooted at x to the root list - remembering to unmark x and update $H_{\min}$ if required - and then *cascade* the cut to y. That is, the process repeats and the subtree rooted at y is also cut and promoted to the root list where we

need to unmark y, but will never need to update $H_{\min}$. The cut continues to cascade upwards until we reach an unmarked node, in which case we mark it and terminate, or until we reach the root of the tree where we also terminate.

Note: In both case $2a$ and $2b$ if the parent of the node we are cutting is ever the root of the tree we never mark this node. If it is already marked than it is not necessary to unmark it.

Example 7: Decrease-key: case 2a

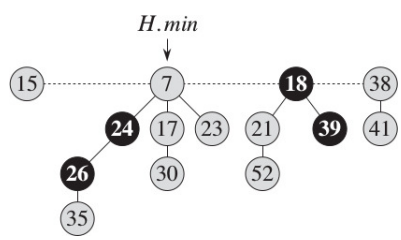
Suppose we want to decrease the key of the node with key 46 to 15 in the heap below.



Decreasing 46 to 15 violates the heap property because the key of its parent is 24 which is greater than 15. To address this violation we cut the subtree rooted at 15 (previously 46) and promote it to the root list.

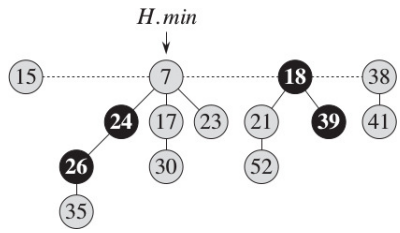
The promotion can be performed by letting the subtree rooted at 15 form a new Fibonacci heap H_1 that can be merged with the current heap adding 15 as the left sibling of 7 and updating $H_{\min}$ if required. However, we would still need to remember to unmark 15 if it was a marked node originally.

Following this promotion we simply mark 24 (the previous parent of 15) as it was unmarked initially and our operation terminates. Notice how by cutting and promoting the offending node we restore our heap order in $O(1)$-time in this case.

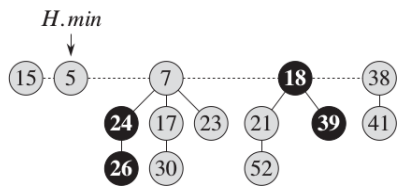


Example 8: Decrease-key: case 2b

Let's now examine the case where the original parent of the subtree we wish to promote is already marked. Suppose we want to decrease the key of the node with key 35 to 5 in the heap that resulted from the decrease-key in example 7.

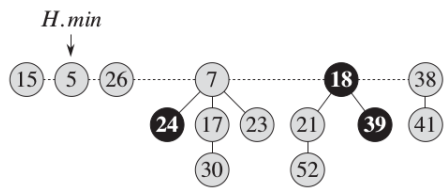


Decreasing 35 to 5 violates the heap property because the key of its parent is 26 which is greater than 5. To address this violation we proceed as we did in example 7 and cut the subtree rooted at 5 (previously 35) and promote it to the root list. We must also remember to unmark 5 if necessary (it's not in this case) and update $H_{\min}$ to point to 5 as it is now the smallest element in the heap as shown below.

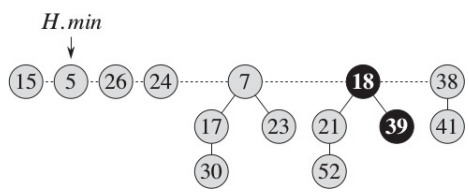


Following this promotion we see that 26 (the previous parent of 5) was marked (indicating it has already lost one child) and so our cut cascades and we repeat the process for 26.

We now cut and promote the subtree rooted at 26 to the root list, being sure to unmark it when doing so.



Then we examine its parent prior to the cut which is another marked node in 24. So once again our cut cascades and we cut and promote 24 to the root list unmarking it as we do so.



Finally, since 7 (the original parent of 24) is a root node our process terminates. Recall that we do not mark root nodes, however if our cascading cut was stopped because we encountered an unmarked node that wasn't the root of the tree then we would mark this node before concluding.

While it is clear that in case 1 and case 2a decrease-key is a constant time operation, some case 2bs can be expensive owing to the cascading cuts. Despite this it can be proven that decrease-key implemented as described above is O(1)-time amortised allowing Dijkstra's to run in $O(|E| + |V|\log|V|)$-time when implemented using a Fibonacci heap as desired. Unfortunately the full amortised analysis is beyond the scope of this document (see week 8) but we will touch on some underlying properties of it in section 3.3.

Question 2: Bounding the degree of nodes in a Fibonacci heap (challenging!)

Show that when decrease-key is implemented the upper bound on the degree of any node in our heap is $\log_\phi(N)$, where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio.

From a different perspective this problem amounts to showing that in the worst case the size of a given tree in H is given by a function that is exponential (with base ϕ) in the degree of the tree, so let's determine how small our trees can get. Let's define a *maximally decreased* tree of degree k to be a tree - of degree k - in a Fibonacci heap that has lost the maximum number of nodes from its subtrees due to decrease-key operations. Put another way, a maximally decreased tree is one from which we can't remove any more nodes via decrease-keys without also decreasing the degree of the root. For instance, the maximally decreased trees of degree 0, 1, 2 and 3 contain 1, 2, 3 and 5 nodes respectively, as shown in figure 3.

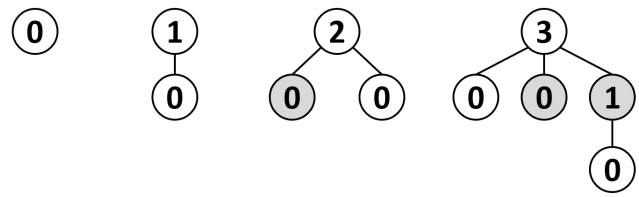


Figure 3: Maximally decreased trees of degree 0, 1, 2 and 3. The marked - denoted by the shading - nodes have lost one child and the numbers represent the degree of each node rather than the key of the node.

With the above in mind you should begin by deriving an expression for the number of nodes in a maximally decreased tree of degree k and then attempt to link this expression to ϕ^k. As a final hint you may find it useful to recall that the Fibonacci sequence is defined as,

$$F_0 = 0, F_1 = 1 \text{ and } F_n = F_{n-1} + F_{n-2} \text{ for } n > 1,$$

and the start of the sequence is shown below.

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}
0	1	1	2	3	5	8	13	21	34	55

3.2.6 Delete

To delete a node from our Fibonacci heap we simply make use of the decrease-key and extract-min operations that we have defined previously. The only point worthy of note here is that just as for decrease-key we must assume that we are passed a reference to the target node x as we cannot efficiently search our Fibonacci heap to find an arbitrary node.

Delete

To delete any node x from a Fibonacci heap H containing N elements:

1. Call decrease-key(x, $-\infty$) to set the key of x to $-\infty$.
2. Run extract-min to remove x from the heap.

The key doesn't necessarily need to be decreased to $-\infty$ as any value that can be treated as less than any of the keys in the heap will also work.

Since delete makes use of decrease-key and extract-min which require $O(1)$ and $O(\log(N))$ time amortised respectively, our delete also requires $O(\log(N))$ time amortised.

3.3 Why mark nodes?

Now that we've discussed how to implement all our operations using a Fibonacci heap let's address the elephant in the room. Why are we marking (and unmarking) nodes?

Practically the purpose of marking the nodes is to keep track of where to make cascading cuts. In other words, they indicate which nodes have already lost one child and so need to be cut and promoted if they lose an additional child. As discussed in section 3.2.5 this ensures that we can maintain the structure of the trees in our heap, which by definition must all be binomial trees where each node is allowed to have at most one child missing. But why was the data structure was defined this way in the first place? For many a satisfactory answer to this question would require delving into the full amortised analysis (see week 8), but we will try to provide some brief insight here.

Firstly, recall that our goal in designing Fibonacci heaps was to improve the time complexity of Dijkstra's from $O(|E|\log|V| + |V|\log|V|)$ to $O(|E| + |V|\log|V|)$. This required us to implement decrease-key in $O(1)$-time (amortised), which we achieved by allowing ourselves the freedom to cut nodes from our trees and promote them to the root list. However, things are rarely free and we need to be careful that when speeding up our decrease-key we don't make any of our other operations worse. In particular, we need to ensure that we don't compromise too much on the effort required to perform extract-min since we still require this to be $O(\log(N))$ amortised. Noting that decrease-keys damage trees and add new nodes to the root list, which increases the cost of future extract-mins, this defines a fundamental tension between two key properties of our data structure. Namely,

1. We need each tree in the Fibonacci heap - although not necessarily a binomial tree - to have size at least exponential in the degree of its root. This helps to ensure that our extract-min can be performed in $O(\log(N))$ (amortised).

2. But we also want the number of cascading cuts in a sequence of heap operations to be bounded by the number of decrease-key/delete operations. This ensures that decrease-key remains $O(1)$ amortised.

Intuitively by cutting a node after it has lost 2 children we strike a balance that maintains these two properties. The purpose of the cascading cuts is to preserve property 1, i.e. we don't want too many nodes to be removed from a tree before we 'recycle' the altered part of the tree to the root list. This ensures that we cannot have more than $O(\log(N))$ trees in our root list following a consolidation and helps to bound the frequency with which we encounter expensive extract-min operations. However, the reason we cut after 2 (and not after just 1) children have been removed is to limit the frequency of the cascading cuts as required by property 2.

3.4 Summary

Table 2 summarises the efficiency with which our heap operations can be implemented using Fibonacci and binary heaps.

Operation	Fibonacci heap	Binary heap
make-new-heap	$O(1)$	$O(1)$
min	$O(1)$	$O(1)$
extract-min	$O(\log N)$ (amortized)	$O(\log N)$
merge	$O(1)$	$O(N)$
decrease-key	$O(1)$ (amortized)	$O(\log N)$
delete	$O(\log N)$ (amortized)	$O(\log N)$
insert	$O(1)$	$O(\log N)$ worst-case $O(1)$ amortized

Table 2: The time complexity of Fibonacci and binary heap operations. The bounds given are worst case bounds unless stated otherwise.